

SalePadel

Decisiones Arquitectónicas (ADRs)

Entregable 4 · Arquitectura y Diseño de Sistemas 2026

Fecha de entrega: 08/06/2026

ID	Título	Estado	Fecha
ADR-001	Elección de la arquitectura del Backend	Aceptada	08/06/2026

1. Contexto

El sistema debe gestionar distintos procesos de negocio relacionados entre sí: registro de usuarios con diferentes roles, administración de canchas, consulta de disponibilidad, reserva de turnos, creación de lobbies y gestión de solicitudes de participación. Estas operaciones involucran reglas de negocio que no deberían quedar mezcladas con la lógica de presentación ni con el acceso directo a la base de datos. Por ejemplo, aceptar una solicitud a un lobby puede requerir validar el estado del lobby, verificar cupos disponibles, actualizar el estado de la solicitud y eventualmente modificar el estado del turno asociado.

Motivador 1 — Mantenibilidad y separación de responsabilidades: el sistema será desarrollado por un equipo y puede crecer en funcionalidades, por lo que se necesita una estructura que permita ubicar claramente dónde se implementan las reglas de negocio, dónde se atienden las solicitudes HTTP y dónde se accede a los datos.

Motivador 2 — Complejidad de las reglas de negocio: operaciones como reservar un turno, crear un lobby o aceptar jugadores no son simples operaciones CRUD, sino procesos que combinan validaciones, cambios de estado y coordinación entre varias entidades del dominio.

Motivador 3 — Evolución del sistema: el proyecto puede incorporar nuevas funcionalidades, como pagos, notificaciones, estadísticas o integración con servicios externos. Por eso se requiere una arquitectura que permita agregar módulos sin modificar de forma excesiva el resto del backend.

2. Alternativas Consideradas

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
Monolito Modular por Capas (Elegida)	Simplicidad, consistencia transaccional y separación lógica estricta que asegura un código escalable y mantenible.	-- (esta fue la elegida)
Microservicios	Escalabilidad y despliegue independiente de cada componente y alta tolerancia a fallos aislados.	"Sobre-diseño" para la escala del sistema. Introduce excesiva complejidad de red, sobrecarga operativa y dificulta enormemente el

		manejo de transacciones complejas.
Monolito tradicional	Implementación simple y rápida.	A medida que crece el sistema, el código tiende a mezclarse y resulta más difícil de mantener.
Arquitectura orientada a eventos	Alto desacoplamiento entre componentes y facilidad para incorporar nuevas funcionalidades reactivas.	Requiere infraestructura adicional y aumenta la complejidad de depuración y control.

3. Decisión Tomada

Se decide una arquitectura de **monolito modular organizado en capas**. La aplicación se despliega como una única unidad, pero el código se organiza en módulos de negocio independientes.

Cada módulo implementa una separación en capas, favoreciendo la mantenibilidad, la reutilización y la separación de responsabilidades.

Fundamentación:

- Razón 1: La organización en capas permite una clara separación de responsabilidades, manteniendo aisladas la lógica de presentación, la lógica de negocio y el acceso a datos. Esto mejora la mantenibilidad del sistema y facilita la localización y modificación del código.
- Razón 2: La complejidad de las reglas de negocio requiere una capa específica de servicios donde implementar validaciones, cambios de estado y coordinación entre múltiples entidades sin acoplar esta lógica a los controladores ni a la persistencia.
- Razón 3: La división en módulos funcionales favorece la evolución futura del sistema, permitiendo incorporar nuevas funcionalidades como pagos, notificaciones o estadísticas de manera organizada y con un impacto reducido sobre los módulos existentes.

4. Consecuencias

Consecuencias Positivas	Trade-offs / Costos
La separación en módulos y capas mejora la organización del código, facilitando su comprensión y mantenimiento. Permite realizar pruebas de integración y <i>End-to-End</i> (E2E) levantando todo el entorno localmente con facilidad. Al utilizar un patrón arquitectónico y un <i>stack</i> tecnológico ya dominado por los	La existencia de múltiples capas implica una mayor cantidad de archivos y código estructural que en una implementación más simple. Todo el backend queda atado al mismo lenguaje y ecosistema, impidiendo usar tecnologías más eficientes para módulos específicos.

desarrolladores, se reduce drásticamente la curva de aprendizaje, acelerando el ciclo de desarrollo inicial y minimizando riesgos técnicos durante la implementación.	Cualquier cambio menor requiere redespregar la aplicación entera, y un error crítico en un solo módulo (ej. un ciclo infinito en pagos) puede comprometer la disponibilidad de todo el backend.
---	---

ID	Titulo	Estado	Fecha
ADR-002	Elección del stack principal para la aplicación web	Aceptada	08/06/2026

1. Contexto
El sistema necesita una aplicación web que permita a distintos tipos de usuarios interactuar con las funcionalidades principales del dominio: gestión de turnos, administración de canchas, creación de lobbies, solicitudes de participación, reservas y consulta de disponibilidad. La solución debe permitir construir una interfaz dinámica, organizar las pantallas del sistema, manejar formularios y consumir datos del backend de forma clara. Además, el equipo necesita una tecnología que facilite el desarrollo dentro de los tiempos del proyecto y que pueda integrarse con las decisiones arquitectónicas del sistema. Motivador 1 — Interfaz dinámica y experiencia de usuario: el sistema requiere pantallas interactivas para consultar disponibilidad, crear reservas, gestionar solicitudes y visualizar estados de turnos o lobbies. Motivador 2 — Velocidad de desarrollo y experiencia del equipo: el equipo necesita trabajar con un stack web conocido, con buena documentación y que permita avanzar rápidamente en la construcción de funcionalidades. Además, algunos integrantes cuentan con experiencia previa en proyectos desarrollados tecnologías del ecosistema web, lo que reduce el riesgo de aprendizaje y facilita la distribución de tareas. Motivador 3 — Organización y despliegue de la aplicación: se busca una solución que permita estructurar el proyecto de forma clara, manejar rutas y facilitar el despliegue de la aplicación web sin incorporar complejidad innecesaria.

2. Alternativas Consideradas		
Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
HTML, CSS y JavaScript puro	Baja complejidad inicial y no requiere framework.	A medida que crecen las pantallas, formularios y estados del sistema, el código se vuelve difícil de mantener. No ofrece una estructura clara para componentes, rutas ni separación de responsabilidades.
Angular	Framework completo, con estructura fuerte, ruteo,	Tiene una curva de aprendizaje mayor y una estructura más rígida. Además, el equipo

	servicios y TypeScript integrados desde el inicio.	posee menor experiencia previa con Angular en comparación con React, lo que puede aumentar el tiempo de desarrollo.
React con Next.js (elegida)	Permite construir la interfaz con componentes React escritos en TSX, aprovechar tipado estático y utilizar una estructura de proyecto más completa para rutas, renderizado, organización y despliegue. El equipo cuenta con experiencia previa en proyectos del ecosistema React, lo que favorece una implementación más rápida.	-- (esta fue la elegida)

3. Decisión Tomada

Se decide utilizar Next.js con TypeScript/TSX como stack principal para la aplicación web del sistema.

Fundamentación:

- Next.js permite construir una aplicación web basada en React, utilizando componentes reutilizables para pantallas como formularios de turnos, vistas de lobbies, listados de canchas, estados de solicitudes y administración de usuarios.
- El uso next como framework ayuda a mejorar el aspecto de calidad de performance de nuestro sistema ya que gracias a que permite el server side rendering, aumenta la velocidad de carga de las páginas.
- Next.js aporta una estructura de aplicación sobre React, facilitando la organización de rutas, pantallas y lógica asociada, reduciendo la cantidad de decisiones técnicas que el equipo debe resolver manualmente.
- La elección favorece la velocidad de desarrollo, ya que el equipo puede implementar funcionalidades web completas dentro de un mismo ecosistema tecnológico, sin separar prematuramente la aplicación en varios proyectos independientes.
- La experiencia previa del equipo en proyectos con React, JavaScript y herramientas web reduce la curva de aprendizaje, permite distribuir tareas con mayor facilidad y disminuye el riesgo de retrasos durante la implementación.

4. Consecuencias	
Consecuencias Positivas	Trade-offs / Costos
Next.js facilita el manejo de rutas y la estructura general de la aplicación.	El proyecto queda acoplado a las convenciones de Next.js. Cambiar de framework en el futuro tendría un costo considerable.
La interfaz puede organizarse mediante componentes reutilizables, reduciendo duplicación en formularios, tarjetas, listados y vistas de estado.	El equipo debe respetar una organización interna clara para evitar que la lógica de negocio quede mezclada con componentes visuales.
La experiencia previa del equipo con React y tecnologías web permite avanzar con mayor rapidez y menor riesgo técnico.	La familiaridad con el stack puede llevar a elegir soluciones conocidas sin evaluar correctamente si siguen siendo adecuadas en futuras etapas.

ID	Título	Estado	Fecha
ADR-003	Comunicación entre componentes	Aceptada	08/06/2026
1. Contexto			
SalePadel tiene tres categorías de comunicación que deben resolverse de forma diferente: (1) la comunicación síncrona entre el frontend Next.js y el backend para operaciones CRUD y consultas, y (2) la comunicación asíncrona para el envío de notificaciones (confirmación de reserva, recordatorios 2hs antes del turno, notificación de fin de partido) y por último (3) la comunicación entre módulos no debe estar acoplada. Motivador 1: Los módulos deben poder evolucionar independientemente. Si el módulo matchmaking cambia su lógica interna, no debe romper el módulo notifications ni obligarlo a modificarse. (Extensibilidad) Motivador 2: Latencia de notificaciones desacoplada: el envío de emails no debe bloquear la respuesta HTTP al usuario. Si el servidor SMTP falla, la operación principal (crear reserva, confirmar lobby) no debe revertirse. Motivador 3: Integridad de las operaciones principales: las acciones críticas del sistema (confirmar lobby, registrar reserva, aceptar solicitud) deben completarse independientemente de sus efectos secundarios. Si el envío de un email o el recálculo de métricas falla, la operación de negocio ya ocurrió y no debe revertirse. Un módulo que llama directamente a otro acopla su transacción al éxito del llamado, un fallo de notificación revertiría la confirmación del lobby. La comunicación desacoplada entre módulos garantiza que cada contexto sea responsable únicamente de su propia transacción.			

2. Alternativas Consideradas – Comunicación síncrona		
Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
Server Actions de Next.js	Integración nativa con React — sin fetch manual, revalidación automática.	Acoplan el frontend al backend en el mismo proceso. No producen endpoints documentables como API REST, lo que imposibilita el entregable de Diseño de APIs. Un cliente móvil futuro no podría consumirlos.
REST HTTP/JSON	Estándar universal, stateless, fácil de documentar con OpenAPI, compatible nativo con Next.js	— (esta fue la elegida)
Alternativas Consideradas – Comunicación asincrona		

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
REST síncrono para todo (incluyendo notificaciones)	Simplicidad total: todo es HTTP, sin infraestructura adicional.	El envío de emails bloquea la respuesta HTTP. Si el SMTP falla, la operación principal falla. Los recordatorios programados (2hs antes) son imposibles de implementar sin un scheduler externo.
Cola de mensajes	Desacopla el envío de emails del flujo principal, soporta reintentos automáticos y jobs programados	Añade excesiva complejidad operativa de configuración, serialización y mantenimiento de un proceso worker activo en el hosting de despliegue.
Eventos in-process (EventEmitter) + Tabla de Persistencia	Desacoplamiento inmediato entre módulos sin infraestructura externa. El fallo del canal de email no bloquea la transacción de negocio. La tabla de notificaciones actúa como registro de estado y soporte para reintentos asíncronos.	— (esta fue la elegida)

Alternativas Consideradas – Comunicación modulo-modulo

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
Llamada directa entre servicios (acoplamiento fuerte)	Simple de implementar — import directo entre módulos.	Viola el principio de inversión de dependencias (DIP) y el aislamiento de bounded contexts. El módulo matchmaking quedaría acoplado al módulo notifications.
EventEmitter in	Cada módulo emite eventos y los módulos interesados se suscriben. El emisor no sabe quién escucha.	— (esta fue la elegida)

3. Decisión Tomada

Comunicación sincrónica:

Se decide: usar REST HTTP/JSON como protocolo de comunicación entre el frontend y el backend.

Fundamentación:

Razón 1 — Produce un contrato HTTP documentable como API REST, cumpliendo el entregable de Diseño de APIs. Los Route Handlers de Next.js son el mecanismo nativo del stack elegido — no requieren setup adicional. El tiempo de respuesta está acotado por el procesamiento síncrono de cada handler, alineado con los tiempos definidos en los CDU.

Razón 2 — Las operaciones del sistema (reservar turno, crear lobby, aceptar solicitud) tienen semántica de comando y respuesta inmediata. REST con los verbos HTTP correctos (POST, PATCH, DELETE) modela directamente esa semántica sin abstracciones adicionales

Comunicación asincrónica:

Se decide: Usar un patrón orientado a eventos mediante el EventEmitter nativo de Node.js, apoyado en una tabla de persistencia en la Base de Datos para asegurar la resiliencia.

Fundamentación:

Razón 1 — El envío de emails no debe condicionar el éxito de las operaciones principales. Cuando se confirma un lobby, la reserva ya es un hecho de negocio consumado; un fallo del servidor SMTP no debe revertirla. La cola desacopla el efecto secundario de la transacción principal.

Razón 2 — Los recordatorios programados 2 horas antes del turno no pueden dispararse desde una petición HTTP.

Comunicación módulo-módulo

Se decide: usar EventEmitter como mecanismo de comunicación entre módulos del backend.

Fundamentación:

Razón 1 — El módulo emisor no debe conocer ni depender de los módulos que reaccionan a sus eventos. Cuando Lobbies confirma un partido, emite un evento lobby.confirmado; son Notificaciones y Evaluaciones quienes se suscriben y actúan. Si se agrega un nuevo módulo que deba reaccionar a ese evento, Lobbies no requiere ninguna modificación.

Razón 2 — Cuando un lobby se confirma, las acciones que se disparan como consecuencia (como enviar notificación, no deben hacer fallar la confirmación si algo sale mal. Con EventEmitter, cada módulo reacciona por su cuenta y si uno falla, el lobby ya estaba confirmado y eso no cambia.

4. Consecuencias

4. Consecuencias	
Consecuencias Positivas	Trade-offs / Costos

(modulo-modulo) Los módulos son independientes: matchmaking no importa a notifications. Cada bounded context puede evolucionar sin romper los demás.	Eventos in-process no son persistentes: si el servidor se reinicia en el momento en que un evento está en vuelo, se pierde. Mitigación: la tabla notificación en BD actúa como registro del estado (pendiente/enviada/fallida).
(sincrona) El contrato REST es documentable y consumible por cualquier cliente HTTP — frontend web, futura app mobile, herramientas de testing.	La disciplina de no importar módulos entre sí debe sostenerse con convenciones de equipo y revisión de código. Sin enforcement automático (ej. ESLint import boundaries), el acoplamiento puede reaparecer.
(asíncrona) Un fallo del servidor SMTP no impacta las operaciones principales. La reserva y la confirmación del lobby son independientes del estado del canal de email.	La cola de reintentos se gestiona a nivel de aplicación en memoria y base de datos relacional, lo que podría consumir más conexiones de PostgreSQL ante picos de carga.

ID	Titulo	Estado	Fecha
ADR-004	Elección de base de datos principal para la capa de persistencia.	Aceptada	08/06/2026

1. Contexto
El sistema SalePadel debe almacenar y consultar información fuertemente relacionada entre sí, incluyendo usuarios, jugadores, reservas, turnos, canchas, lobbies y solicitudes de ingreso. Muchas de las operaciones principales requieren navegar relaciones entre múltiples entidades, por ejemplo: obtener los lobbies de un jugador, verificar disponibilidad de turnos, gestionar solicitudes de ingreso o generar reportes de reservas y utilización de canchas. Motivador 1 - Consistencia transaccional: Las operaciones de reserva y organización de partidos involucran múltiples entidades relacionadas. Es necesario garantizar que cambios como reservar un turno, crear un lobby o aceptar una solicitud se ejecutan de forma consistente para evitar estados inválidos o datos contradictorios. Motivador 2 - Modelo de datos altamente relacional: El dominio contiene numerosas relaciones entre entidades (Usuario, Jugador, Turno, Reserva, Lobby y Solicitud), por lo que las consultas frecuentes requieren joins y restricciones de integridad referencial. Motivador 3 - Experiencia del equipo: El equipo tiene experiencia con bases de datos relacionales y con SQL, pero ningún miembro posee experiencia previa con bases de datos no relacionales.

2. Alternativas Consideradas		
Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
MongoDB	Esquema flexible y rápida evolución de documentos.	El dominio posee relaciones complejas y consultas que requieren joins. La gestión de integridad y consistencia recaería en la aplicación, aumentando la complejidad.
MySQL	Base de datos relacional madura, ampliamente utilizada y con soporte ACID.	Menor flexibilidad en tipos avanzados y funcionalidades modernas respecto de PostgreSQL.
PostgreSQL + Neon	Soporte completo de transacciones ACID, joins	— (esta fue la elegida)

	complejos, restricciones de integridad, tipos avanzados y despliegue administrado en la nube mediante Neon.	
--	---	--

3. Decisión Tomada

Se decide: utilizar PostgreSQL como base de datos principal del sistema, desplegada mediante Neon como proveedor de infraestructura administrada.

Fundamentación:

1. El modelo de datos de SalePadel es inherentemente relacional, con múltiples entidades vinculadas mediante claves foráneas y consultas que requieren joins eficientes entre usuarios, reservas, turnos, lobbies y solicitudes.
2. PostgreSQL proporciona transacciones ACID completas que permiten mantener la consistencia de operaciones críticas como reservas de turnos, creación de lobbies y aceptación de solicitudes de ingreso.
3. La experiencia previa del equipo con SQL y bases de datos relacionales reduce la curva de aprendizaje, disminuye riesgos de implementación y acelera el desarrollo. Además, Neon simplifica el despliegue y administración de la infraestructura de persistencia.

4. Consecuencias

Consecuencias Positivas	Trade-offs / Costos
Las relaciones entre entidades pueden modelarse directamente mediante claves foráneas y restricciones de integridad.	El esquema es más rígido que en una base NoSQL; cambios estructurales requieren migraciones.
Las operaciones críticas de reserva y gestión de partidos pueden implementarse utilizando transacciones ACID.	La escalabilidad horizontal es más limitada que en algunas soluciones NoSQL distribuidas.
Prisma se integra de forma natural con PostgreSQL, simplificando el acceso a datos.	Dependencia de un proveedor externo (Neon) para la infraestructura de persistencia.
Neon elimina la necesidad de administrar servidores de base de datos propios y facilita el despliegue inicial.	El uso de un servicio administrado puede generar costos crecientes si aumenta significativamente la carga del sistema.

ID	Título	Estado	Fecha
ADR-005	Elección del Sistema de Despliegue	Aceptada	08/06/2026

1. Contexto

El sistema debe ser accesible a través de Internet y contar con un mecanismo de despliegue que facilite las actualizaciones, reduzca la carga operativa sobre el equipo y permita una integración sencilla con las tecnologías utilizadas en el proyecto. Además, se busca minimizar el tiempo dedicado a tareas de infraestructura y administración de servidores.

Motivador 1: por simplicidad operativa el equipo dispone de tiempo y recursos limitados para administrar infraestructura propia (performance, costo, experiencia del equipo).

Motivador 2: la aplicación está desarrollada con Next.js y requiere un entorno compatible para evitar configuraciones engorrosas de enrutamiento.

Motivador 3: se busca automatizar el proceso de despliegue y facilitar la publicación de nuevas versiones.

2. Alternativas Consideradas

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
Vercel (PaaS)	Despliegue automatizado e integración nativa con Next.js. Plan gratuito completo para el alcance de nuestro proyecto.	-- (esta fue elegida)
Microsoft Azure	Muy fáciles de utilizar para desplegar aplicaciones backend.	Alta complejidad, requiere conocimientos avanzados de Cloud Computing. Además, la capa gratuita es muy corta y restrictiva y su modelo de facturación es complejo y elevado para el proyecto.
Servidor propio	Mayor control sobre la infraestructura.	Requiere configuración, mantenimiento, monitoreo y actualizaciones manuales.
Contenedores Docker en la nube	Mayor control y portabilidad del entorno de ejecución.	Mayor complejidad operativa para el alcance actual del proyecto (configuración compleja).

3. Decisión Tomada

Se decide utilizar Vercel (Plan Gratuito) como plataforma principal de despliegue para la aplicación.

Fundamentación:

Razón 1: Vercel ofrece integración nativa con Next.js, facilitando el despliegue de páginas, solucionando el motivador 2.

Razón 2: La plataforma permite automatizar los despliegues a partir del repositorio, reduciendo el esfuerzo operativo y acelerando la entrega de nuevas versiones, solucionando el motivador 3.

Razón 3: el equipo tiene experiencia en Vercel debido a su uso en la materia Ingeniería de Aplicaciones Web; además, al usar el plan gratuito que ofrece el costo es nulo.

4. Consecuencias

Consecuencias Positivas	Trade-offs / Costos
Despliegues automáticos y sencillos. Integración directa con Next.js y GitHub. El plan gratuito (Hobby) soporta holgadamente el volumen de tráfico esperado.	Menor control sobre la infraestructura. Dependencia de un proveedor externo. La aplicación queda parcialmente dependiente de Vercel, por lo que un cambio de proveedor podría requerir trabajo adicional de migración.

ID	Título	Estado	Fecha
ADR-006	Elección de ORM para el manejo de la BD	Aceptada	08/06/2026

1. Contexto

El sistema requiere una capa de acceso a datos que permita interactuar con PostgreSQL de forma segura y mantenible. El modelo de datos incluye múltiples relaciones entre usuarios, reservas, turnos, lobbies y solicitudes, por lo que es necesario simplificar la escritura de consultas y el mantenimiento del esquema.

Motivador 1: Reducir la cantidad de SQL manual y errores asociados.

Motivador 2: Facilitar la evolución del esquema mediante migraciones controladas.

Motivador 3: Integración con el stack tecnológico elegido (Next.js y TypeScript).

2. Alternativas Consideradas

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
SQL nativo	Máximo control sobre las consultas y optimización.	Mayor cantidad de código repetitivo y mantenimiento manual del esquema.
TypeORM	ORM maduro y ampliamente utilizado en el ecosistema Node.js.	Configuración más compleja y menor seguridad de tipos en comparación con Prisma. Usa decoradores, por lo que no conecta directamente con TypeScript.
Prisma	Cliente tipado, migraciones integradas y excelente integración con TypeScript.	— (esta fue la elegida)

3. Decisión Tomada

Se decide: utilizar Prisma como ORM y herramienta de acceso a datos para interactuar con PostgreSQL.

Fundamentación:

1. Prisma genera automáticamente un cliente tipado que reduce errores al interactuar con la base de datos.
2. Su sistema de migraciones facilita la evolución controlada del esquema de datos.
3. Se integra de forma natural con Next.js, TypeScript y PostgreSQL, tecnologías ya seleccionadas para el proyecto.

4. Consecuencias	
Consecuencias Positivas	Trade-offs / Costos
Menor cantidad de código de acceso a datos.	Dependencia de una herramienta adicional dentro de la arquitectura.
Mayor seguridad de tipos durante el desarrollo.	Algunas consultas muy específicas pueden requerir SQL nativo.
Migraciones y mantenimiento del esquema simplificados.	

ID	Título	Estado	Fecha
ADR-007	Integración de API de Mapas	Aceptada	08/06/2026

1. Contexto

El sistema necesita mostrar la ubicación del complejo deportivo a los jugadores para que puedan llegar al lugar donde tienen reservado su turno.

Motivador 1: La solución debe ser completamente gratuita y sin riesgo de corte de servicio por superar límites de uso, ya que el equipo no dispone de presupuesto para APIs pagas ni tarjeta de crédito registrada en plataformas de cobro por uso.

Motivador 2: La funcionalidad requerida es puntual y acotada: mostrar un único marcador con la ubicación fija del complejo. No se necesitan rutas, geocoding dinámico ni mapas personalizados.

Motivador 3: La integración debe ser compatible con el stack existente (Next.js) sin agregar infraestructura adicional ni dependencias de servicios externos que requieran gestión de API keys o rotación de secretos.

2. Alternativas Consideradas

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
Google Maps API	Mayor precisión y SDK completo	Requiere tarjeta de crédito; pago al superar el crédito gratuito mensual
Mapbox	Buen diseño visual, 50.000 cargas gratuitas/mes	Límite mensual con cobro posterior; requiere API key y cuenta
HERE Maps	Plan gratuito más generoso (250.000 transacciones/mes)	SDK poco documentado en el ecosistema Next.js; curva de aprendizaje mayor
Leaflet + OpenStreetMap	100% gratuito, sin cuenta, sin límites, open source	— (esta fue la elegida)

3. Decisión Tomada

Se decide: usar Leaflet como librería de mapas en el frontend, consumiendo tiles de OpenStreetMap como fuente de datos cartográficos, para mostrar la ubicación fija del complejo deportivo.

Fundamentación:

Razón 1: Es la única alternativa que es completamente gratuita sin condiciones: no requiere cuenta, no tiene límites de requests y no puede generar costos inesperados, resolviendo directamente el motivador de presupuesto.

Razón 2: La funcionalidad necesaria es mostrar un único marcador en una coordenada fija. Leaflet cubre ese caso con pocas líneas de código y sin necesidad de llamadas a una API externa desde el backend, lo que simplifica la arquitectura.

Razón 3: Leaflet se integra en Next.js mediante el paquete `react-leaflet`, que tiene documentación activa y es compatible con el stack del proyecto sin configuración adicional de infraestructura ni gestión de secretos.

4. Consecuencias

Consecuencias Positivas	Trade-offs / Costos
Sin riesgo de corte de servicio por límites de uso o vencimiento de crédito gratuito.	Los tiles de OpenStreetMap son provistos por servidores comunitarios; su disponibilidad no está garantizada por un SLA formal (aunque para uso académico es aceptable).
No se expone ninguna API key en el frontend ni se requiere gestión de secretos para esta funcionalidad.	El detalle cartográfico de OpenStreetMap en zonas poco mapeadas puede ser inferior al de Google Maps, aunque para mostrar una dirección urbana es suficiente.
La librería Leaflet corre completamente en el cliente, sin necesidad de un endpoint en el backend para esta funcionalidad.	Al no usar un backend para los mapas, el endpoint GET <code>/canchas/{id}/ubicacion</code> del ADR de APIs devuelve solo las coordenadas almacenadas en base de datos; el renderizado del mapa es responsabilidad exclusiva del frontend.

ID	Título	Estado	Fecha
ADR-008	Elección de manejo de autenticación.	Aceptada	08/06/2026

1. Contexto

El sistema necesita autenticar usuarios para permitir el acceso seguro a sus funcionalidades. Existen distintos tipos de usuarios, como jugadores, administradores o responsables de la gestión de canchas y turnos, por lo que resulta necesario identificar de manera confiable quién accede al sistema y qué acciones puede realizar. La autenticación incluye procesos sensibles como registro de usuarios, inicio de sesión, cierre de sesión, manejo de sesiones activas, recuperación de cuenta y protección de rutas privadas. Además, el backend debe poder validar la identidad del usuario autenticado antes de permitir operaciones sobre entidades del dominio, como reservas, turnos, lobbies o solicitudes de participación.

Motivador 1 — Seguridad y reducción de riesgos: la autenticación es una funcionalidad crítica, y una implementación incorrecta puede comprometer la seguridad del sistema y de los datos de los usuarios.

Motivador 2 — Integración con el stack tecnológico: el sistema se desarrolla utilizando Next.js, React y TypeScript, por lo que se necesita una solución que se integre correctamente tanto con el frontend como con el backend.

Motivador 3 — Separación entre autenticación y lógica de dominio: el sistema necesita diferenciar la identidad del usuario de la información propia del negocio.

2. Alternativas Consideradas

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
Clerk	Integración directa con Next.js, gestión de sesiones, componentes de autenticación listos para usar y experiencia previa del equipo.	— (esta fue la elegida)
Autenticación propia	Máximo control sobre usuarios, sesiones, credenciales y reglas de autenticación.	Aumenta considerablemente la complejidad y el riesgo de errores de seguridad. Requiere implementar almacenamiento seguro de contraseñas, recuperación de cuenta, manejo de sesiones y validación de tokens.
Firebase Authentication	Servicio administrado de autenticación con soporte	Introduce una dependencia fuerte con el ecosistema

	para múltiples proveedores de inicio de sesión.	Firestore/Google, mientras que el sistema ya está orientado a Next.js, PostgreSQL y Prisma.
--	---	---

3. Decisión Tomada

Se decide utilizar Clerk como servicio externo de autenticación del sistema. sesión, cierre de sesión, sesiones activas y obtención del usuario autenticado, a su vez que la metadata que contendrá el rol del usuario. El sistema no almacenará contraseñas en su base de datos. En su lugar, almacenará únicamente información propia del dominio, como perfiles internos, datos de jugador, administrador o responsable de gestión.

Fundamentación:

- Razón 1: Clerk permite delegar una responsabilidad crítica de seguridad en un proveedor especializado, evitando implementar manualmente registro, login, recuperación de contraseña, manejo de sesiones y almacenamiento seguro de credenciales.
- Razón 2: Clerk se integra adecuadamente con Next.js, que es el framework utilizado en el proyecto. Esto facilita proteger rutas, obtener el usuario autenticado desde el backend y manejar sesiones dentro del mismo stack tecnológico.
- Razón 3: El equipo ya cuenta con experiencia previa utilizando Clerk en otros proyectos. Esto reduce la curva de aprendizaje, acelera la implementación y disminuye el riesgo de errores durante el desarrollo.

4. Consecuencias

Consecuencias Positivas	Trade-offs / Costos
Se reduce el tiempo necesario para implementar autenticación.	El sistema pasa a depender de un proveedor externo para una funcionalidad crítica.
Se evita almacenar contraseñas en la base de datos propia del sistema.	Un cambio futuro de proveedor requeriría adaptar la integración y migrar identificadores externos de usuario.
Se facilita la protección de rutas privadas y endpoints del backend.	Es necesario mantener correctamente sincronizados los usuarios de Clerk con los perfiles internos del sistema.
Se separa la autenticación de la lógica de negocio, manteniendo el dominio más limpio.	El modelo de datos interno queda acoplado al uso de un identificador externo, como clerkUserId.

ID	Título	Estado	Fecha
ADR-009	Integración de API de Clima	Aceptada	08/06/2026

1. Contexto

El sistema requiere mostrar información meteorológica asociada a las reservas y turnos. Para ello es necesario obtener datos actualizados de fuentes externas, ya que la aplicación no dispone de información climática propia. Se debe definir un mecanismo para consumir estos datos y manejar posibles fallos o limitaciones del servicio externo, sobre las condiciones climáticas y mostrarlos en la aplicación cuando sea relevante para la gestión de reservas y turnos.

Motivador 1: La información climática cambia constantemente, por lo que no resulta viable mantener estos datos dentro del sistema.

Motivador 2: Se busca una solución con documentación clara y una API sencilla de consumir desde la aplicación.

Motivador 3: La solución debe contar con un plan gratuito o de bajo costo que cubra las necesidades actuales del proyecto y permita crecer en el futuro.

2. Alternativas Consideradas

Alternativa	Ventaja Principal	Desventaja/Motivo de Descarte
OpenWeather	Amplia documentación, plan gratuito y datos meteorológicos completos.	— (esta fue la elegida)
Open-Meteo	Gratuita y de fácil acceso.	Menor cantidad de funcionalidades complementarias y ejemplos de integración.
WeatherAPI	API simple de utilizar y con buena documentación.	Límites restrictivos en el plan gratuito.

3. Decisión Tomada

Se decide: OpenWeather como proveedor externo de información meteorológica para consultar las condiciones climáticas asociadas a las reservas y turnos.

Fundamentación:

Razón 1: OpenWeather ofrece la información meteorológica necesaria para el proyecto mediante una API ampliamente utilizada y bien documentada.

Razón 2: El plan gratuito resulta suficiente para el volumen de consultas esperado durante el desarrollo y uso inicial del sistema.

Razón 3: La disponibilidad de documentación, ejemplos y bibliotecas compatibles facilita la integración y el mantenimiento de la solución.

--

4. Consecuencias	
Consecuencias Positivas	Trade-offs / Costos
Acceso a información meteorológica actualizada y confiable.	Dependencia de un servicio externo para obtener los datos climáticos.
Mejora de la experiencia del usuario al brindar información relevante para la planificación de reservas.	El proveedor puede imponer límites de uso o modificar sus políticas en el futuro.